

/start Python



```
/fix <code>
```

```
/quick_fix <code>
```

```
/explain <code>
```

```
/review <code>
```

```
/search <query>
```

```
/help
```



Build FastAPI using python for a chatbot where I can talk by voice, upload all types of extensions of documents and ask about the uploaded data, you can use free models from openrouter .





1. Setup FastAPI app
2. File Upload Endpoint:
 - Accept files
 - Extract text based on file type
 - Chunk text
 - Generate embeddings
 - Store in vector DB (FAISS)
3. Voice Endpoint:
 - Accept audio file
 - Convert speech to text (Whisper)
4. Chat Endpoint:
 - Accept user query
 - Embed query
 - Retrieve top-k relevant chunks
 - Build prompt (context + question)
 - Call OpenRouter API
 - Return answer
5. Optional:
 - Text-to-speech response



```
app/  
├─ main.py  
├─ rag.py  
├─ utils.py  
├─ config.py  
└─ requirements.txt
```



app/config.py

```
import os  
  
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
```

```
OPENROUTER_MODEL = "mistralai/mistral-7b-instruct:free"

EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
VECTOR_DB_PATH = "faiss_index"
```

app/utils.py

```
from pydub import AudioSegment
import whisper
import tempfile

model = whisper.load_model("base")

def speech_to_text(file):
    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:
        audio = AudioSegment.from_file(file.file)
        audio.export(tmp.name, format="wav")
        result = model.transcribe(tmp.name)
    return result["text"]
```

app/rag.py

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.document_loaders import PyPDFLoader, TextLoader, Docx2txtLoader
import os
from app.config import EMBEDDING_MODEL, VECTOR_DB_PATH

embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)

def load_document(file_path):
    if file_path.endswith(".pdf"):
        loader = PyPDFLoader(file_path)
    elif file_path.endswith(".docx"):
        loader = Docx2txtLoader(file_path)
    else:
        loader = TextLoader(file_path)
    return loader.load()
```

```
def process_and_store(file_path):
    docs = load_document(file_path)

    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_documents(docs)

    if os.path.exists(VECTOR_DB_PATH):
        db = FAISS.load_local(VECTOR_DB_PATH, embeddings)
        db.add_documents(chunks)
    else:
        db = FAISS.from_documents(chunks, embeddings)

    db.save_local(VECTOR_DB_PATH)

def retrieve(query):
    db = FAISS.load_local(VECTOR_DB_PATH, embeddings)
    return db.similarity_search(query, k=4)
```

app/main.py

```
from fastapi import FastAPI, UploadFile, File, Form
import shutil
import os
import requests

from app.rag import process_and_store, retrieve
from app.utils import speech_to_text
from app.config import OPENROUTER_API_KEY, OPENROUTER_MODEL

app = FastAPI()

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    file_path = os.path.join(UPLOAD_DIR, file.filename)

    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    process_and_store(file_path)
```

```
        return {"message": "File uploaded and indexed successfully"}

@app.post("/voice")
async def voice_input(file: UploadFile = File(...)):
    text = speech_to_text(file)
    return {"transcript": text}

@app.post("/chat")
async def chat(query: str = Form(...)):
    docs = retrieve(query)
    context = "\n".join([doc.page_content for doc in docs])

    prompt = f"""
    Answer based only on context:
    {context}

    Question: {query}
    """

    response = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json"
        },
        json={
            "model": OPENROUTER_MODEL,
            "messages": [{"role": "user", "content": prompt}]
        }
    )

    return response.json()
```

requirements.txt

```
fastapi
uvicorn
python-multipart
langchain
faiss-cpu
sentence-transformers
pypdf
docx2txt
whisper
pydub
requests
```





```
pip install -r requirements.txt  
export OPENROUTER_API_KEY=your_key_here  
  
uvicorn app.main:app --reload
```

<http://127.0.0.1:8000/docs>



Ok, do the upgrades and recommended improvements.



1. Add Auth middleware (API key check)
2. Memory:
 - Store session_id → chat history
 - Append messages per session
3. Streaming:
 - Use OpenRouter stream=True
 - Yield tokens via generator
4. WebSocket:
 - Real-time chat endpoint
 - Handle session + streaming
5. TTS:
 - Convert final response → audio (gTTS)
6. Refactor:
 - Separate services (LLM, memory, rag)
 - Clean architecture
7. Docker:
 - Dockerfile + run config



```
app/  
├─ main.py  
├─ rag.py  
└─ utils.py
```



```
|— config.py
|— memory.py
|— llm.py
|— auth.py
|— tts.py
|— requirements.txt
└─ Dockerfile
```



app/config.py

```
import os

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_MODEL = "mistralai/mistral-7b-instruct:free"

EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
VECTOR_DB_PATH = "faiss_index"

APP_API_KEY = os.getenv("APP_API_KEY", "secret123")
```

app/auth.py

```
from fastapi import Header, HTTPException
from app.config import APP_API_KEY

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != APP_API_KEY:
        raise HTTPException(status_code=401, detail="Unauthorized")
```

app/memory.py

```
from collections import defaultdict
```

```
memory_store = defaultdict(list)

def get_history(session_id: str):
    return memory_store[session_id]

def add_message(session_id: str, role: str, content: str):
    memory_store[session_id].append({"role": role, "content": content})
```

app/llm.py

```
import requests
from app.config import OPENROUTER_API_KEY, OPENROUTER_MODEL

def stream_chat(messages):
    response = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json"
        },
        json={
            "model": OPENROUTER_MODEL,
            "messages": messages,
            "stream": True
        },
        stream=True
    )

    for line in response.iter_lines():
        if line:
            decoded = line.decode("utf-8")
            if "data:" in decoded:
                yield decoded.replace("data: ", "")
```

app/tts.py

```
from gtts import gTTS
import uuid
import os
```

```
AUDIO_DIR = "audio"
os.makedirs(AUDIO_DIR, exist_ok=True)

def text_to_speech(text: str):
    filename = f"{uuid.uuid4()}.mp3"
    path = os.path.join(AUDIO_DIR, filename)

    tts = gTTS(text)
    tts.save(path)

    return path
```

app/main.py

```
from fastapi import FastAPI, UploadFile, File, Form, Depends, WebSocket
from fastapi.responses import StreamingResponse
import shutil
import os

from app.rag import process_and_store, retrieve
from app.utils import speech_to_text
from app.memory import get_history, add_message
from app.llm import stream_chat
from app.auth import verify_api_key
from app.tts import text_to_speech

app = FastAPI()

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/upload", dependencies=[Depends(verify_api_key)])
async def upload_file(file: UploadFile = File(...)):
    path = os.path.join(UPLOAD_DIR, file.filename)
    with open(path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    process_and_store(path)
    return {"message": "uploaded"}

@app.post("/voice", dependencies=[Depends(verify_api_key)])
async def voice(file: UploadFile = File(...)):
    text = speech_to_text(file)
```

```
    return {"text": text}

@app.post("/chat", dependencies=[Depends(verify_api_key)])
async def chat(session_id: str = Form(...), query: str = Form(...)):
    history = get_history(session_id)

    docs = retrieve(query)
    context = "\n".join([d.page_content for d in docs])

    messages = history + [
        {"role": "user", "content": f"Context:\n{context}\n\n0: {query}"}
    ]

    def generate():
        full_response = ""

        for chunk in stream_chat(messages):
            full_response += chunk
            yield chunk

        add_message(session_id, "user", query)
        add_message(session_id, "assistant", full_response)

    return StreamingResponse(generate(), media_type="text/plain")

@app.post("/tts", dependencies=[Depends(verify_api_key)])
async def tts(text: str = Form(...)):
    path = text_to_speech(text)
    return {"audio_path": path}

@app.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()

    session_id = "ws_session"

    while True:
        data = await ws.receive_text()

        docs = retrieve(data)
        context = "\n".join([d.page_content for d in docs])

        messages = get_history(session_id) + [
            {"role": "user", "content": f"{context}\n\n{data}"}
        ]

        full = ""

        for chunk in stream_chat(messages):
```

```
        full += chunk
        await ws.send_text(chunk)

    add_message(session_id, "user", data)
    add_message(session_id, "assistant", full)
```

requirements.txt

```
fastapi
uvicorn
python-multipart
langchain
faiss-cpu
sentence-transformers
pypdf
docx2txt
whisper
pydub
requests
gtts
```



Dockerfile

```
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install --no-cache-dir -r app/requirements.txt

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```



```
docker build -t chatbot-api .
docker run -p 8000:8000 -e OPENROUTER_API_KEY=xxx chatbot-api
```



Ok, do the next level. Don't forget to create unit tests for all features



1. Replace FAISS → Qdrant
 - Use local container
 - Store embeddings
2. Backend:
 - Refactor RAG → Qdrant client
 - Add service layer



- Add pytest tests
3. Frontend (React):
 - Chat UI
 - WebSocket streaming
 - File upload
 - Voice record + send
 4. Docker Compose:
 - API
 - Qdrant
 - Frontend
 5. Tests:
 - Upload
 - Chat
 - Voice
 - TTS
 - Auth



```
project/
├─ app/
├─ frontend/
├─ tests/
├─ docker-compose.yml
└─ Dockerfile
```



app/rag.py

```
from langchain_qdrant import Qdrant
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders import PyPDFLoader, TextLoader, Docx2txtLoader
from qdrant_client import QdrantClient
import os

EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"

client = QdrantClient(host="qdrant", port=6333)
```

```
embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)

COLLECTION = "docs"

def load_doc(path):
    if path.endswith(".pdf"):
        return PyPDFLoader(path).load()
    elif path.endswith(".docx"):
        return Docx2txtLoader(path).load()
    return TextLoader(path).load()

def store(path):
    docs = load_doc(path)
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_documents(docs)

    Qdrant.from_documents(
        chunks,
        embeddings,
        url="http://qdrant:6333",
        collection_name=COLLECTION,
    )

def search(query):
    db = Qdrant(
        client=client,
        collection_name=COLLECTION,
        embeddings=embeddings,
    )
    return db.similarity_search(query, k=4)
```



frontend/src/App.jsx

```
import { useState, useEffect, useRef } from "react";

export default function App() {
    const [messages, setMessages] = useState([]);
    const [input, setInput] = useState("");
    const ws = useRef(null);

    useEffect(() => {
```



```

ws.current = new WebSocket("ws://localhost:8000/ws");

ws.current.onmessage = (event) => {
  setMessages((prev) => [...prev, { role: "assistant", text: event.data }]);
};

return () => ws.current.close();
}, []);

const sendMessage = () => {
  ws.current.send(input);
  setMessages((prev) => [...prev, { role: "user", text: input }]);
  setInput("");
};

return (
  <div style={{ padding: 20 }}>
    <h1>Chatbot</h1>

    <div>
      {messages.map((m, i) => (
        <div key={i}>
          <b>{m.role}</b> {m.text}
        </div>
      ))}
    </div>

    <input value={input} onChange={(e) => setInput(e.target.value)} />
    <button onClick={sendMessage}>Send</button>
  </div>
);
}

```



tests/test_api.py

```

import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

API_KEY = {"x-api-key": "secret123"}

```

```
def test_upload():
    with open("tests/sample.txt", "w") as f:
        f.write("hello world")

    with open("tests/sample.txt", "rb") as f:
        response = client.post(
            "/upload",
            files={"file": ("sample.txt", f)},
            headers=API_KEY,
        )

    assert response.status_code == 200

def test_chat():
    response = client.post(
        "/chat",
        data={"session_id": "test", "query": "hello"},
        headers=API_KEY,
    )
    assert response.status_code == 200

def test_tts():
    response = client.post(
        "/tts",
        data={"text": "hello"},
        headers=API_KEY,
    )
    assert response.status_code == 200

def test_auth_fail():
    response = client.post("/chat", data={"session_id": "x", "query": "hi"})
    assert response.status_code == 401
```



docker-compose.yml

```
version: "3.9"

services:
  api:
    build: .
    ports:
```

```
- "8000:8000"
environment:
  - OPENROUTER_API_KEY=your_key
depends_on:
  - qdrant

qdrant:
  image: qdrant/qdrant
  ports:
    - "6333:6333"

frontend:
  build: ./frontend
  ports:
    - "3000:3000"
```



```
pytest -v
```



Do the final upgrade ideas



1. Add PostgreSQL (users table)
2. Implement JWT auth:
 - signup
 - login
 - token verification
3. Store sessions in DB
4. Add LangGraph:
 - Define tools (RAG search, calculator)
 - Create agent graph
 - Execute per query
5. Update chat endpoint:
 - Use agent instead of direct LLM
6. Add tests:
 - auth
 - agent



```
app/  
├─ auth.py  
├─ db.py  
└─ models.py
```

```
|— schemas.py  
|— agent.py
```



app/db.py

```
from sqlalchemy import create_engine  
from sqlalchemy.orm import sessionmaker, declarative_base  
  
DATABASE_URL = "postgresql://user:password@db:5432/chatbot"  
  
engine = create_engine(DATABASE_URL)  
SessionLocal = sessionmaker(bind=engine)  
  
Base = declarative_base()
```

app/models.py

```
from sqlalchemy import Column, Integer, String  
from app.db import Base  
  
class User(Base):  
    __tablename__ = "users"  
  
    id = Column(Integer, primary_key=True, index=True)  
    username = Column(String, unique=True)  
    password = Column(String)
```

app/schemas.py

```
from pydantic import BaseModel  
  
class UserCreate(BaseModel):
```

```
username: str
password: str

class UserLogin(BaseModel):
    username: str
    password: str
```



app/auth.py

```
from datetime import datetime, timedelta
from jose import jwt
from fastapi import HTTPException, Depends
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from app.db import SessionLocal
from app.models import User

SECRET = "secret"
ALGO = "HS256"

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def create_token(data: dict):
    return jwt.encode(
        {**data, "exp": datetime.utcnow() + timedelta(hours=12)},
        SECRET,
        algorithm=ALGO,
    )

def verify_token(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=[ALGO])
```

```
        return payload
    except:
        raise HTTPException(status_code=401, detail="Invalid token")
```



app/agent.py

```
from langgraph.graph import StateGraph
from langchain.tools import tool
from app.rag import search
import requests
from app.config import OPENROUTER_API_KEY

@tool
def doc_search(query: str):
    docs = search(query)
    return "\n".join([d.page_content for d in docs])

@tool
def calculator(expr: str):
    try:
        return str(eval(expr))
    except:
        return "error"

def call_llm(messages):
    r = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
        json={"model": "mistralai/mistral-7b-instruct:free", "messages": messages},
    )
    return r.json()["choices"][0]["message"]["content"]

def build_agent():
    graph = StateGraph(dict)

    def node(state):
        query = state["input"]

        if "calculate" in query:
            result = calculator(query.replace("calculate", ""))
```

```

        else:
            result = doc_search(query)

            answer = call_llm(
                [{"role": "user", "content": f"{result}\n\nQuestion: {query}"}]
            )

            return {"output": answer}

graph.add_node("agent", node)
graph.set_entry_point("agent")

return graph.compile()

agent = build_agent()

```



app/main.py

```

from app.agent import agent
from app.auth import verify_token

@app.post("/chat")
async def chat(query: str, user=Depends(verify_token)):
    result = agent.invoke({"input": query})
    return {"response": result["output"]}

```



tests/test_auth.py

```

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_signup_login():
    client.post("/signup", json={"username": "u", "password": "p"})

```



```
r = client.post("/login", data={"username": "u", "password": "p"})
assert r.status_code == 200
```



```
version: "3.9"
```

```
services:
```

```
  api:
```

```
    build: .
```

```
    ports:
```

```
      - "8000:8000"
```

```
    depends_on:
```

```
      - db
```

```
      - qdrant
```

```
  db:
```

```
    image: postgres
```

```
    environment:
```

```
      POSTGRES_USER: user
```

```
      POSTGRES_PASSWORD: password
```

```
      POSTGRES_DB: chatbot
```

```
    ports:
```

```
      - "5432:5432"
```

```
  qdrant:
```

```
    image: qdrant/qdrant
```

```
    ports:
```

```
      - "6333:6333"
```

```
  frontend:
```

```
    build: ./frontend
```

```
    ports:
```

```
      - "3000:3000"
```





Yeah, do the elite level



1. Add Redis:

- Cache RAG results
- Store sessions
- Rate limiting

2. Extend tools:

- Google search tool
- Generic API caller
- Improve calculator safety

3. Upgrade agent:

- Tool routing logic
- Multi-step reasoning loop

4. Middleware:

- Rate limiting per IP/user



5. Tests:

- Redis cache
- Rate limit
- Tools



```
app/  
├─ redis_client.py  
├─ rate_limit.py  
└─ tools.py
```



app/redis_client.py

```
import redis  
import os  
  
redis_client = redis.Redis(  
    host=os.getenv("REDIS_HOST", "redis"),  
    port=6379,  
    decode_responses=True  
)
```



app/rate_limit.py

```
from fastapi import Request, HTTPException  
from app.redis_client import redis_client  
  
LIMIT = 20 # requests/min  
  
async def rate_limiter(request: Request):
```

```
ip = request.client.host
key = f"rate:{ip}"

count = redis_client.incr(key)

if count == 1:
    redis_client.expire(key, 60)

if count > LIMIT:
    raise HTTPException(status_code=429, detail="Rate limit exceeded")
```



app/tools.py

```
import requests
import ast
from app.redis_client import redis_client

def cached(key, fn):
    if redis_client.exists(key):
        return redis_client.get(key)

    result = fn()
    redis_client.setex(key, 300, result)
    return result

def safe_eval(expr: str):
    try:
        tree = ast.parse(expr, mode="eval")

        for node in ast.walk(tree):
            if not isinstance(node, (
                ast.Expression, ast.BinOp, ast.UnaryOp,
                ast.Num, ast.Load, ast.operator
            )):
                return "Invalid expression"

        return str(eval(expr))
    except:
        return "Error"

def google_search(query: str):
```

```
def fetch():
    url = f"https://serpapi.com/search?q={query}&api_key=YOUR_SERPAPI_KEY"
    return requests.get(url).text

return cached(f"google:{query}", fetch)

def call_api(url: str):
    def fetch():
        return requests.get(url).text

    return cached(f"api:{url}", fetch)
```



app/agent.py

```
from app.tools import google_search, call_api, safe_eval
from app.rag import search
import requests
from app.config import OPENROUTER_API_KEY

def call_llm(messages):
    r = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
        json={"model": "mistralai/mistral-7b-instruct:free", "messages": messages},
    )
    return r.json()["choices"][0]["message"]["content"]

def route(query: str):
    if "http" in query:
        return call_api(query)
    if "search" in query:
        return google_search(query.replace("search", ""))
    if any(op in query for op in ["+", "-", "*", "/"]):
        return safe_eval(query)
    return "\n".join([d.page_content for d in search(query)])

def agent(query: str):
    context = route(query)

    return call_llm(
```

```
[{"role": "user", "content": f"{context}\n\nQuestion: {query}"}]
```



app/main.py

```
from app.rate_limit import rate_limiter

@app.post("/chat", dependencies=[Depends(rate_limiter)])
async def chat(query: str, user=Depends(verify_token)):
    return {"response": agent(query)}
```



tests/test_rate_limit.py

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_rate_limit():
    for _ in range(25):
        r = client.post("/chat", params={"query": "hi"})
        assert r.status_code == 429
```

tests/test_tools.py

```
from app.tools import safe_eval

def test_calc():
    assert safe_eval("2+2") == "4"
```



```
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
      - qdrant
      - redis

  db:
    image: postgres
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: chatbot

  qdrant:
    image: qdrant/qdrant

  redis:
    image: redis

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
```





Yeah, I want both options



1. Kubernetes:
 - Deploy API, frontend, Redis, Qdrant, Postgres
 - Add Services + Ingress
 - Add HPA autoscaling
2. Observability:
 - Add Prometheus metrics endpoint
 - Add Grafana dashboards
 - Add OpenTelemetry tracing
3. Instrument FastAPI:
 - Request latency
 - Errors
 - Throughput
4. Docker updates



```
k8s/api-deployment.yaml
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: chatbot-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: chatbot-api
  template:
    metadata:
      labels:
        app: chatbot-api
    spec:
      containers:
        - name: api
          image: chatbot-api:latest
          ports:
            - containerPort: 8000
          env:
            - name: OPENROUTER_API_KEY
              value: "your_key"
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: chatbot-api
spec:
  selector:
    app: chatbot-api
  ports:
    - port: 80
      targetPort: 8000
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: chatbot-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
name: chatbot-api
minReplicas: 2
maxReplicas: 10
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: chatbot-ingress
spec:
  rules:
    - host: chatbot.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: chatbot-api
                port:
                  number: 80
```



```
pip install prometheus-fastapi-instrumentator opentelemetry-sdk opentelemetry-instrum
```

```
app/observability.py
```

```
from prometheus_fastapi_instrumentator import Instrumentator
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider

trace.set_tracer_provider(TracerProvider())

def setup_metrics(app):
    Instrumentator().instrument(app).expose(app)
```

main.py

```
from app.observability import setup_metrics

setup_metrics(app)
```



k8s/prometheus.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    scrape_configs:
      - job_name: 'chatbot'
        static_configs:
          - targets: ['chatbot-api:80']
```



k8s/grafana.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      labels:
        app: grafana
    spec:
      containers:
        - name: grafana
          image: grafana/grafana
          ports:
            - containerPort: 3000
```



observability.py

```
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

def setup_tracing(app):
    FastAPIInstrumentor.instrument_app(app)
```

main.py

≡ Code Copilot  Free offer



```
from app.observability import setup_tracing

setup_tracing(app)
```



```
RUN pip install prometheus-fastapi-instrumentator opentelemetry-sdk opentelemetry-in
```



```
kubectl apply -f k8s/
```



I want both





1. CI/CD:
 - On push → test → build → push image
 - Deploy to Kubernetes
2. Multi-region:
 - Deploy clusters (us, eu, asia)
 - Use global load balancer
3. CDN:
 - Cache frontend + API responses
 - Route users to nearest region
4. Edge:
 - Cloudflare Workers
 - Optional inference at edge



```
.github/workflows/deploy.yml
```

```
name: Deploy

on:
  push:
    branches: [main]

jobs:
  test-build-deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Install Python
        uses: actions/setup-python@v4
        with:
          python-version: 3.10

      - name: Install deps
        run: pip install -r app/requirements.txt

      - name: Run tests
```

```
run: pytest -v

- name: Build Docker
  run: docker build -t chatbot-api .

- name: Login to DockerHub
  run: echo "${{ secrets.DOCKER_PASS }}" | docker login -u "${{ secrets.DOCKER_USER }}" --password-stdin

- name: Push image
  run: |
    docker tag chatbot-api "${{ secrets.DOCKER_USER }}/chatbot-api:latest"
    docker push "${{ secrets.DOCKER_USER }}/chatbot-api:latest"

- name: Deploy to K8s
  run: |
    kubectl set image deployment/chatbot-api api=${{ secrets.DOCKER_USER }}/chatbot-api:latest
```



Users

↓

Cloudflare CDN (global)

↓

Geo Routing

↓

K8s Cluster (US / EU / ASIA)

↓

API Pods (autoscaled)



```
kubectl config use-context us-cluster
kubectl apply -f k8s/
```

```
kubectl config use-context eu-cluster  
kubectl apply -f k8s/
```



worker.js

```
export default {  
  async fetch(request) {  
    const country = request.cf.country;  
  
    let origin = "https://us.api.com";  
  
    if (country === "DE") origin = "https://eu.api.com";  
    if (country === "IN") origin = "https://asia.api.com";  
  
    return fetch(origin, request);  
  }  
};
```



/static/*

/tts/*

/chat

/auth



```
strategy:  
  type: RollingUpdate  
  rollingUpdate:  
    maxUnavailable: 0  
    maxSurge: 1
```



```
DOCKER_USER  
DOCKER_PASS  
KUBE_CONFIG  
OPENROUTER_API_KEY
```





Now return me everything as a zipped folder to download and run locally using docker desktop



.....





```
docker compose up --build
```



a



```
docker compose up --build
```

```
http://localhost:8000
```



Are you able to create a GitHub repository and commit everything to it ?





ai-chatbot-platform

```
# unzip project
unzip full_chatbot_project.zip
cd full_chatbot_project

git init
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/ai-chatbot-platform.git

git add .
git commit -m "initial commit"

git push -u origin main
```





gh



Generate full GitHub repo version



ai-chatbot-platform/



```
|— app/
|   |— main.py
|   |— config.py
|   |— auth.py
|   |— db.py
|   |— models.py
|   |— schemas.py
|   |— rag.py
```

```
| ├── agent.py  
| ├── tools.py  
| ├── redis_client.py  
| ├── rate_limit.py  
| ├── memory.py  
| ├── llm.py  
| ├── tts.py  
| ├── observability.py  
| └── requirements.txt  
  
|── frontend/  
|   ├── package.json  
|   └── src/App.jsx  
  
|── tests/  
|   ├── test_api.py  
|   ├── test_auth.py  
|   ├── test_tools.py  
|   └── test_rate_limit.py  
  
|── k8s/  
|   ├── api-deployment.yaml  
|   ├── hpa.yaml  
|   ├── ingress.yaml  
|   ├── prometheus.yaml  
|   └── grafana.yaml  
  
|── .github/workflows/deploy.yml  
|── docker-compose.yml  
|── Dockerfile  
└── README.md
```



app/main.py

```
from fastapi import FastAPI, Depends, UploadFile, File, Form  
from fastapi.responses import StreamingResponse  
from app.auth import verify_token  
from app.agent import agent  
from app.rate_limit import rate_limiter  
from app.observability import setup_metrics, setup_tracing  
  
app = FastAPI()  
  
setup_metrics(app)
```

```
setup_tracing(app)

@app.get("/")
def root():
    return {"status": "ok"}

@app.post("/chat", dependencies=[Depends(rate_limiter)])
async def chat(query: str = Form(...), user=Depends(verify_token)):
    return {"response": agent(query)}
```

app/config.py

```
import os

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
SECRET = os.getenv("JWT_SECRET", "secret")
```

app/redis_client.py

```
import redis

redis_client = redis.Redis(host="redis", port=6379, decode_responses=True)
```

app/tools.py

```
import ast

def safe_eval(expr: str):
    try:
        tree = ast.parse(expr, mode="eval")
        return str(eval(expr))
    except:
        return "error"
```


app/agent.py

```
from app.tools import safe_eval
import requests
from app.config import OPENROUTER_API_KEY

def agent(query: str):
    if any(op in query for op in ["+", "-", "*", "/"]):
        return safe_eval(query)

    r = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
        json={
            "model": "mistralai/mistral-7b-instruct:free",
            "messages": [{"role": "user", "content": query}]
        }
    )

    return r.json()["choices"][0]["message"]["content"]
```

frontend/src/App.jsx

```
import { useState } from "react";

export default function App() {
  const [msg, setMsg] = useState("");
  const [out, setOut] = useState("");

  const send = async () => {
    const res = await fetch("http://localhost:8000/chat", {
      method: "POST",
      body: new URLSearchParams({ query: msg }),
    });
    const data = await res.json();
    setOut(data.response);
  };

  return (
    <div>
      <h1>AI Chatbot</h1>
      <input value={msg} onChange={e=>setMsg(e.target.value)} />
    </div>
  );
}
```

</> JavaScript



docker-compose.yml

```
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - OPENROUTER_API_KEY=your_key
    depends_on:
      - redis
      - db
      - qdrant

  redis:
    image: redis

  db:
    image: postgres
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: chatbot
```

```
qdrant:
  image: qdrant/qdrant

frontend:
  build: ./frontend
  ports:
    - "3000:3000"
```

`.github/workflows/deploy.yml`

```
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Install deps
        run: pip install -r app/requirements.txt

      - name: Run tests
        run: pytest

      - name: Build Docker
        run: docker build -t chatbot .

      - name: Push & Deploy (stub)
        run: echo "deploy here"
```

`README.md`

```
# AI Chatbot Platform

## Run locally
docker compose up --build
```

```
## API  
http://localhost:8000/docs
```



```
git init  
git remote add origin https://github.com/YOUR_USER/ai-chatbot-platform.git  
git add .  
git commit -m "full platform"  
git push -u origin main
```



ok, send me all files as zip;





```
unzip ai_chatbot_platform.zip  
cd ai_chatbot_platform  
docker compose up --build
```

<http://localhost:8000>



yeah, both options

